

TÉCNICAS DE PROGRAMAÇÃO 1

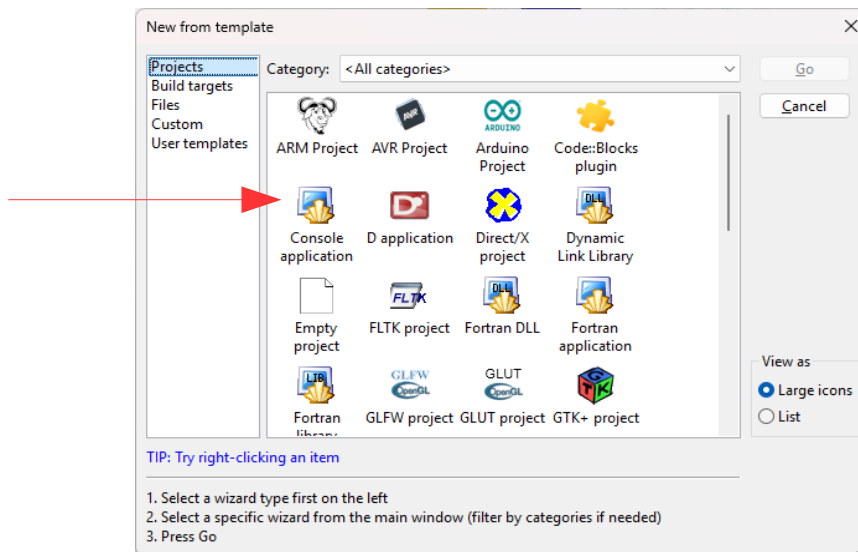
FERNANDO ALBUQUERQUE

LABORATÓRIO 1

ROTEIRO DE ATIVIDADES

1. Criar projeto **Console application** em C++ usando o Code::Blocks:

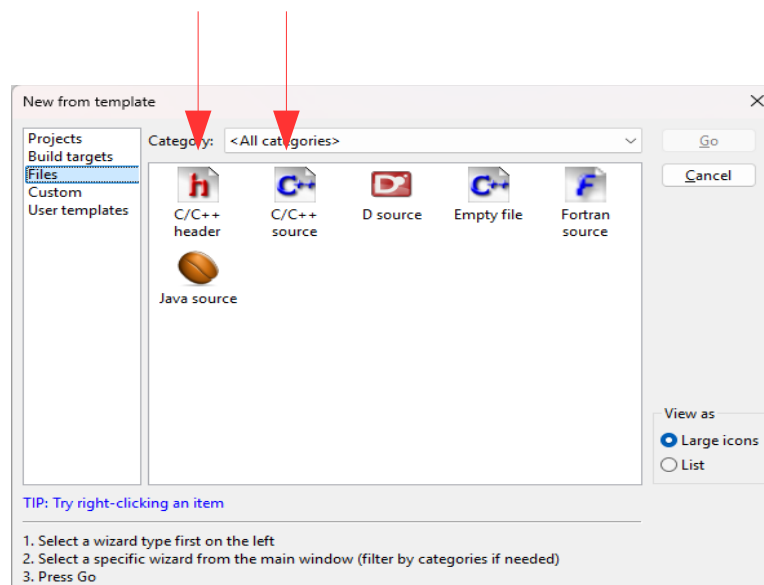
File → New → Project



2. Criar arquivos **dominios.cpp** e **dominios.hpp**.

File → New → File... Selecionar C/C++ **header** e informar nome do arquivo **dominios.hpp**

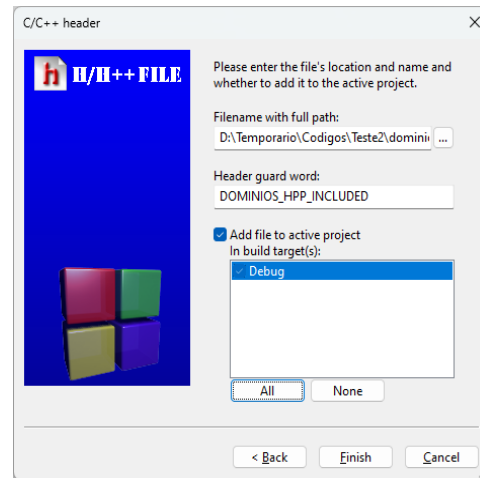
File → New → File... Selecionar C/C++ **source** e informar nome do arquivo **dominios.cpp**



Selecionar **Add file to active project in build target(s)**

Selecionar **All**

Selecionar **Finish**



3. Compilar e executar **Hello world!**

Build → Build and run

```
Hello world!  
Process returned 0 (0x0)   execution time : 0.052 s  
Press any key to continue.
```

4. Declarar classe domínio no arquivo **dominios.hpp**.

```
using namespace std;  
  
class CodigoCliente {  
    private:  
        static const int LIMITE = 5;  
        int valor;  
        bool validar(int);  
    public:  
        bool setValor(int);  
        int getValor();  
};  
  
inline int CodigoCliente::getValor() {  
    return valor;  
}
```

5. Implementar métodos no arquivo **dominios.cpp**.

```
#include "dominios.hpp"  
  
bool CodigoCliente::validar(int valor) {  
    if (valor > LIMITE)  
        return false;  
    return true;  
}  
  
bool CodigoCliente::setValor(int valor) {  
    if (!validar(valor))  
        return false;
```

```

        this->valor = valor;
        return true;
    }

```

6. Complementar o código da função **main** de modo a possibilitar a correta execução e compilação.

```

#include <iostream>
#include "dominios.hpp"

using namespace std;

int main() {

    // Criar objeto da classe domínio.

    // Ler valor a partir da entrada padrão.

    // Armazenar valor no objeto.

    // Se o valor for válido, apresentar valor armazenado no objeto.

    // Se o valor for inválido, informar que o valor não foi armazenado.

    return 0;
}

```

7. Complementar o código da função **main** de modo a possibilitar a correta execução e compilação.

```

#include <iostream>
#include "dominios.hpp"

using namespace std;

int main() {

    // Criar ponteiro para objeto da classe domínio.

    // Criar objeto e armazenar o seu identificador no ponteiro.

    // Ler valor a partir da entrada padrão.

    // Armazenar valor no objeto.

    // Se o valor for válido, apresentar valor armazenado no objeto.

    // Se o valor for inválido, informar que o valor não foi armazenado.

    // Destruir objeto apontado.

    return 0;
}

```

8. Codifique uma das classes domínio presentes na especificação do trabalho.

LABORATÓRIO 2

ROTEIRO DE ATIVIDADES

1. Reusar o projeto do laboratório anterior.
2. Criar arquivos **entidades.cpp** e **entidades.hpp**.

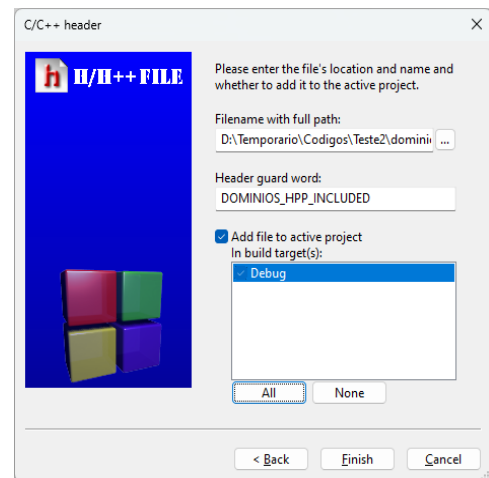
File → New → File... Selecionar C/C++ **header** e informar nome do arquivo **entidades.hpp**

File → New → File... Selecionar C/C++ **source** e informar nome do arquivo **entidades.cpp**

Selecionar **Add file to active project in build target(s)**

Selecionar **All**

Selecionar **Finish**



3. Declarar as classes domínio no arquivo **dominios.hpp** e implementar os métodos **inline**.

```
class Codigo {
private:
    static const int LIMITE = 25;
    int valor;
    bool validar(int);
public:
    bool setValor(int);
    int getValor();
};

inline int Codigo::getValor(){
    return valor;
}

class Prioridade {
private:
    static const int INVALIDO = 0;
    int valor;
    bool validar(int);
public:
    bool setValor(int);
    int getValor();
};
```

```

inline int Prioridade::getValor(){
    return valor;
}

```

4. Implementar métodos das classes domínio no arquivo **dominios.cpp**.

```

bool Codigo::validar(int codigo){
    if (codigo > LIMITE)
        return false;
    return true;
}

bool Codigo::setValor(int valor){
    if (!validar(valor))
        return false;
    this->valor = valor;
    return true;
}

bool Prioridade::validar(int valor){
    if (valor == INVALIDO)
        return false;
    return true;
}

bool Prioridade::setValor(int valor) {
    if (!validar(valor))
        return false;
    this->valor = valor;
    return true;
}

```

5. Declarar classe entidade no arquivos **entidades.hpp** e implementar os métodos **inline**.

```

#include "dominios.hpp"

using namespace std;

class Projeto {
private:
    Codigo    codigo;
    Prioridade prioridade;
public:
    void setCodigo(Codigo);
    Codigo getCodigo();
    void setPrioridade(Prioridade);
    Prioridade getPrioridade();
};

inline void Projeto::setCodigo(Codigo codigo){
    this->codigo = codigo;
}

inline Codigo Projeto::getCodigo() {
    return codigo;
}

inline void Projeto::setPrioridade(Prioridade prioridade){
    this->prioridade = prioridade;
}

```

```
inline Prioridade Projeto::getPrioridade() {  
    return prioridade;  
}
```

6. Complementar o código da função **main** de modo a possibilitar a correta execução e compilação.

```
#include "entidades.hpp"  
  
using namespace std;  
  
int main() {  
    // Criar objetos das classes domínio.  
  
    // Ler valores a partir da entrada padrão.  
  
    // Armazenar valores nos objetos da classes domínio criados.  
  
    // Criar objeto da classe entidade.  
  
    // Armazenar objetos das classes domínio no objeto da classe entidade criado.  
  
    // Ler objetos armazenados na classe domínio e apresentar valor armazenado em cada um desses objetos.  
  
    return 0;  
}
```

7. Codifique uma das classes entidade presentes na especificação do trabalho.

LABORATÓRIO 3

ROTEIRO DE ATIVIDADES

1. Reusar o projeto do laboratório anterior.
2. **Refatorar as classes domínio** de modo a organizá-las em uma **hierarquia de classes**.
3. **Projetar e construir classe base domínio** adaptando a seguinte estrutura:

CÓDIGO EM DOMINIOS.HPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS E TIPOS

// Adaptar trecho de código para cada classe domínio.

```
class Dominio {
    private:
        Tipo valor;                // Atributo para armazenar valor.
        virtual bool validar(Tipo) = 0; // Método abstrato.
    public:
        bool setValor(Tipo);        // Método para atribuir valor.
        Tipo getValor() const;      // Método para recuperar valor.
};

inline Tipo Dominio::getValor() const { // Implementação de método.
    return valor;
}
```

CÓDIGO EM DOMINIOS.CPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS E TIPOS

```
#include "dominios.hpp"

bool Dominio::setValor(Tipo valor) {
    if (!validar(valor))
        return false;
    this->valor = valor;
    return true;
}
```

4. **Projetar e construir classes domínio derivadas da classe base** adaptando a seguinte estrutura:

CÓDIGO EM DOMINIOS.HPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS E TIPOS

```
class DominioA:public Dominio {
    private:
        bool validar(Tipo); // Método para validar valor.
};
```

CÓDIGO EM DOMINIOS.CPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS E TIPOS

```
bool DominioA::validar(int valor){
```

```
        // Implementar código de validação.

        return true;
    }
}
```

5. Complementar o código da função **main** de modo a possibilitar a correta execução e compilação.

```
#include <iostream>
#include "dominios.hpp"

using namespace std;

int main() {

    // Criar objeto da classe domínio.

    // Ler valor a partir da entrada padrão.

    // Armazenar valor no objeto.

    // Se o valor for válido, apresentar valor armazenado no objeto.

    // Se o valor for inválido, informar que o valor não foi armazenado.

    return 0;
}
```


LABORATÓRIO 4

ROTEIRO DE ATIVIDADES

1. Reusar o projeto do laboratório anterior.
2. Refatorar códigos das classes domínio de modo a usar exceções adaptando a seguinte estrutura:

CÓDIGO EM DOMINIOS.HPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS E TIPOS

```
#include <stdexcept>

using namespace std;

// Adaptar trecho de código para cada classe domínio.

class Dominio {
private:
    Tipo valor;                // Atributo para armazenar valor.
    void validar(Tipo);        // Método para validar valor.
public:
    void setValor(Tipo);        // Método para atribuir valor.
    Tipo getValor() const;     // Método para recuperar valor.
};

inline Tipo Dominio::getValor() const {    // Implementação de método.
    return valor;
}
```

CÓDIGO EM DOMINIOS.CPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS E TIPOS

```
#include "dominios.hpp"

void Dominio::validar(Tipo valor){

    // Implementar código de validação.

    // Lançar exceção quando o valor for inválido.

    throw invalid_argument("Argumento invalido.");

}

void Dominio::setValor(Tipo valor) {
    validar(valor);
    this->valor = valor;
}
```

3. Complementar o código da função **main** de modo a possibilitar a correta execução e compilação.

```
#include <iostream>
#include <stdexcept>
#include "dominios.hpp"
```

```
using namespace std;

int main() {

    // Criar objeto da classe domínio.

    // Ler valor a partir da entrada padrão.

    // Armazenar valor no objeto.

    // Se o valor for válido, apresentar valor armazenado no objeto.

    // Se o valor for inválido, capturar exceção e informar que o valor não foi armazenado.

    return 0;

}
```

4. Codifique uma das classes domínio presentes na especificação do trabalho usando exceções.

LABORATÓRIO 5

ROTEIRO DE ATIVIDADES

1. Reusar o projeto do laboratório anterior.
2. Criar arquivos **testes.cpp** e **testes.hpp**.

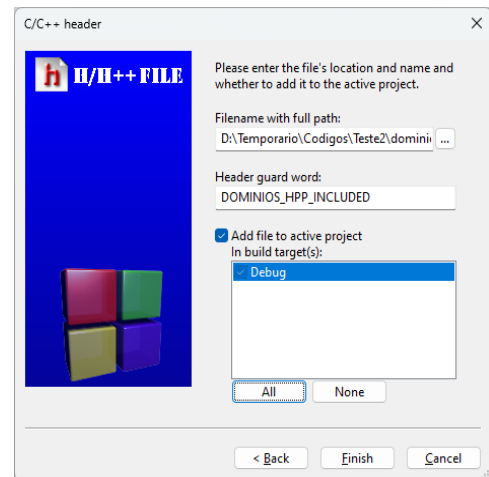
File → New → File... Selecionar C/C++ **header** e informar nome do arquivo **testes.hpp**

File → New → File... Selecionar C/C++ **source** e informar nome do arquivo **testes.cpp**

Selecionar **Add file to active project in build target(s)**

Selecionar **All**

Selecionar **Finish**



3. **Projetar e construir testes de unidade de domínio** adaptando o código:

CÓDIGO EM TESTES.HPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS

```
#include <stdexcept>
#include "dominios.hpp"
```

```
using namespace std;
```

```
class TUDominio {
private:
    const static int VALOR_VALIDO = 20; // Definição de constante para evitar número mágico.
    const static int VALOR_INVALIDO = 30; // Definição de constante para evitar número mágico.
    Dominio *dominio; // Referência para unidade em teste.
    int estado; // Estado do teste.
    void setUp(); // Método para criar unidade em teste.
    void tearDown(); // Método para destruir unidade em teste.
    void testarCenarioValorValido(); // Cenário de teste com valor válido.
    void testarCenarioValorInvalido(); // Cenário de teste com valor inválido.
public:
    const static int SUCESSO = 0; // Definição de constante para reportar resultado.
    const static int FALHA = -1; // Definição de constante para reportar resultado.
    int run(); // Método para executar teste.
};
```

CÓDIGO EM TESTES.CPP

ATENÇÃO PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE TESTES E DOMÍNIOS

```
#include "testes.hpp"

void TUDominio::setUp(){
    dominio = new Dominio();
    estado = SUCESSO;
}

void TUDominio::tearDown(){
    delete dominio;
}

void TUDominio::testarCenarioValorValido(){
    try{
        dominio->setValor(VALOR_VALIDO);
        if (dominio->getValor() != VALOR_VALIDO)
            estado = FALHA;
    }
    catch(invalid_argument &excecao){
        estado = FALHA;
    }
}

void TUDominio::testarCenarioValorInvalido(){
    try{
        dominio->setValor(VALOR_INVALIDO);
        estado = FALHA;
    }
    catch(invalid_argument &excecao){
        if (dominio->getValor() == VALOR_INVALIDO)
            estado = FALHA;
    }
}

int TUDominio::run(){
    setUp();
    testarCenarioValorValido();
    testarCenarioValorInvalido();
    tearDown();
    return estado;
}
```

4. Complementar o código da função **main** de modo a possibilitar a correta execução e compilação.

```
#include <iostream>
#include <stdexcept>
#include "dominios.hpp"
#include "testes.hpp"

using namespace std;

int main() {

    // Criar objeto da classe de teste.

    // Executar teste.

    // Apresentar resultado da execução do teste.
```

```
        return 0;  
    }
```

5. Codifique teste de unidade para uma das classes domínio presentes na especificação do trabalho.

LABORATÓRIO 6

ROTEIRO DE ATIVIDADES

1. Reusar o projeto do laboratório anterior.
2. **Projetar e construir testes de unidade de entidade** adaptando o código:

CÓDIGO EM TESTES.HPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE TESTES E ENTIDADES.

```
#include "dominios.hpp"
#include "entidades.hpp"

class TUEntidade {
private:
    const static int VALOR_VALIDO = 20; // Definição de constante para evitar número mágico.
    Entidade *entidade; // Referência para unidade em teste.
    int estado; // Estado do teste.
    void setUp(); // Método para criar unidade em teste.
    void tearDown(); // Método para destruir unidade em teste.
    void testarCenario(); // Cenário de teste.
public:
    const static int SUCESSO = 0; // Definição de constante para reportar resultado.
    const static int FALHA = -1; // Definição de constante para reportar resultado.
    int run(); // Método para executar teste.
};
```

CÓDIGO EM TESTES.CPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE TESTES E DOMÍNIOS

```
#include "testes.hpp"

void TUEntidade::setUp(){
    entidade = new Entidade();
    estado = SUCESSO;
}

void TUEntidade::tearDown(){
    delete entidade;
}

void TUEntidade::testarCenario(){
    // Repetir trecho de código para cada domínio na entidade em teste.

    Dominio dominio;
    dominio.setValor(VALOR_VALIDO);
    entidade->setNomeAtributo(dominio);
    if(entidade->getNomeAtributo().getValor() != VALOR_VALIDO)
        estado = FALHA;
}

int TUEntidade::run(){
    setUp();
    testarCenario();
    tearDown();
}
```

```
        return estado;  
    }
```

3. Complementar o código da função **main** de modo a possibilitar a correta execução e compilação.

```
#include <iostream>  
#include "testes.hpp"  
  
using namespace std;  
  
int main() {  
  
    // Criar objeto da classe de teste.  
  
    // Executar teste.  
  
    // Apresentar resultado da execução do teste.  
  
    return 0;  
}
```

4. Codifique teste de unidade para uma das classes entidade presentes na especificação do trabalho.

LABORATÓRIO 7

ROTEIRO DE ATIVIDADES

1. Criar projeto **Console application** em C++ usando o Code::Blocks:

File → New → Project

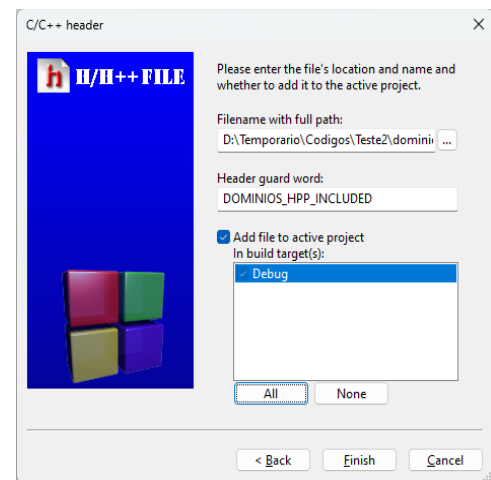
2. Criar arquivos **dominios.hpp**, **entidades.hpp**, **dominios.cpp** e **entidades.cpp**.

File → New → File... Selecionar C/C++ **header** e informar nome do arquivo

Selecionar **Add file to active project in build target(s)**

Selecionar **All**

Selecionar **Finish**



3. Construir os seguintes códigos nos arquivos **.hpp** e **.cpp** adequados.

```
#include <iostream>
#include <string>
#include <stdexcept>

using namespace std;

// Classe Nome
// Value Object
// Formato: 3 a 10 caracteres e compostos por letras.

class Nome {
private:
    string valor;
    bool validarLetra(char) const;
    void validar(const string&) const;
public:
    explicit Nome(const string&);
    string getValor() const;
    bool operator==(const Nome&) const;
};

inline bool Nome::validarLetra(char c) const {
    return (c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z');
}
```



```

void Nome::validar(const string& nome) const {
    unsigned int tamanho = nome.length();

    if (tamanho < 3 || tamanho > 10) {
        throw invalid_argument("Formato invalido");
    }

    for (unsigned int i = 0; i < tamanho; ++i) {
        if (!validarLetra(nome[i])) {
            throw invalid_argument("Formato invalido");
        }
    }
}

Nome::Nome(const string& nome) {
    validar(nome);
    valor = nome;
}

inline string Nome::getValor() const {
    return valor;
}

inline bool Nome::operator==(const Nome& nome) const {
    return this->valor == nome.valor;
}

// Classe Codigo
// Value Object
// Tamanho de 6 caracteres, 3 letras seguidas de 3 dígitos.

class Codigo {
private:
    string valor;
    bool validarLetra(char) const;
    bool validarDigito(char) const;
    void validar(const string&) const;
public:
    explicit Codigo(const string&);
    string getValor() const;
    bool operator==(const Codigo&) const;
};

inline bool Codigo::validarLetra(char c) const {
    return (c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z');
}

inline bool Codigo::validarDigito(char c) const {
    return (c >= '0' && c <= '9');
}

void Codigo::validar(const string& codigo) const {
    if (codigo.length() != 6) {
        throw invalid_argument("Formato invalido");
    }

    for (int i = 0; i < 3; ++i) {
        if (!validarLetra(codigo[i])) {
            throw invalid_argument("Formato invalido");
        }
    }
    if (!validarDigito(codigo[i + 3])) {
        throw invalid_argument("Formato invalido");
    }
}

```

```

    }
}

Codigo::Codigo(const string& codigo) {
    validar(codigo);
    valor = codigo;
}

inline string Codigo::getValor() const {
    return valor;
}

inline bool Codigo::operator==(const Codigo& codigo) const {
    return valor == codigo.valor;
}

// Classe Usuario
// Classe entidade
// Composta por nome e código.

class Usuario {
private:
    Nome nome;
    Codigo codigo;

public:
    Usuario(const Nome&, const Codigo&);
    string getCodigo() const;
    string getNome() const;
    void setNome(const Nome&);
    bool operator==(const Usuario&) const;
};

Usuario::Usuario(const Nome& nome, const Codigo& codigo) : nome(nome), codigo(codigo) {}

inline string Usuario::getNome() const {
    return nome.getValor();
}

inline string Usuario::getCodigo() const {
    return codigo.getValor();
}

inline void Usuario::setNome(const Nome& nome) {
    this->nome = nome;
}

inline bool Usuario::operator==(const Usuario& usuario) const {
    return this->codigo == usuario.codigo;
}

```

4. Complementar o código da função **main** de modo a possibilitar a correta execução e compilação.

```

int main() {

    // Ler valor a partir da entrada padrão.

    // Criar objeto da classe Nome usando o valor lido.

    // Se o valor for válido, apresentar valor armazenado no objeto.

    // Se o valor for inválido, capturar exceção e informar que o valor não foi armazenado.

```

```

// Ler valor a partir da entrada padrão.

// Criar objeto da classe Codigo usando o valor lido.

// Se o valor for válido, apresentar valor armazenado no objeto.

// Se o valor for inválido, capturar exceção e informar que o valor não foi armazenado.

// Ler dois valores.

// Criar dois objetos da classe Nome usando os valores lidos.

// Usar o operador == para comparar os objetos da classe Nome que foram criados.

// Apresentar o resultado da comparação.

// Ler dois valores.

// Criar dois objetos da classe Codigo usando os valores lidos.

// Usar o operador == para comparar os objetos da classe Codigo que foram criados.

// Apresentar o resultado da comparação.

// Criar dois objetos da classe Usuario usando os objetos anteriormente criados.

// Usar o operador == para comparar os objetos da classe Usuario que foram criados.

// Apresentar resultado da comparação.

return 0;
}

```

5. Converta uma das classes domínio especificadas no trabalho para objeto valor (*object value*).

LABORATÓRIO 8

ROTEIRO DE ATIVIDADES

1. Criar projeto **Console application** em C++ usando o Code::Blocks:

File → New → Project

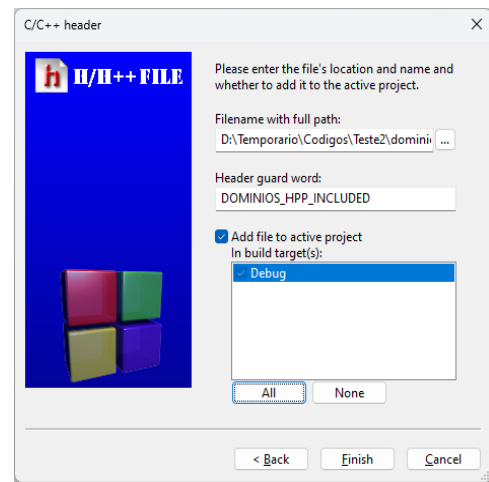
2. Criar arquivos **dominios.hpp**, **fabricas.hpp**, **dominios.cpp** e **fabricas.cpp**.

File → New → File... Selecionar C/C++ header e informar nome do arquivo

Selecionar **Add file to active project in build target(s)**

Selecionar **All**

Selecionar **Finish**



3. Em **dominios.hpp** e **dominios.cpp** construir **classe base Dominio** adaptando a seguinte estrutura:

CÓDIGO EM DOMINIOS.HPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS E TIPOS

```
using namespace std;
```

```
// Adaptar trecho de código para cada classe domínio.
```

```
class Dominio {  
    private:  
        Tipo valor;                // Atributo para armazenar valor.  
        virtual bool validar(Tipo) = 0; // Método abstrato.  
    public:  
        bool setValor(Tipo);        // Método para atribuir valor.  
        Tipo getValor() const;      // Método para recuperar valor.  
};  
  
inline Tipo Dominio::getValor() const {    // Implementação de método.  
    return valor;  
}
```

CÓDIGO EM DOMINIOS.CPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS E TIPOS

```
#include "dominios.hpp"

bool Dominio::setValor(Tipo valor) {
    if (!validar(valor))
        return false;
    this->valor = valor;
    return true;
}
```

4. Construir **quatro classes domínio derivadas da classe base** adaptando a seguinte estrutura:

CÓDIGO EM DOMINIOS.HPP

ATENSTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS E TIPOS

```
class DominioA1:public Dominio {
private:
    bool validar(Tipo);    // Método para validar valor.
};

class DominioB1:public Dominio {
private:
    bool validar(Tipo);    // Método para validar valor.
};

class DominioA2:public Dominio {
private:
    bool validar(Tipo);    // Método para validar valor.
};

class DominioB2:public Dominio {
private:
    bool validar(Tipo);    // Método para validar valor.
};
```

CÓDIGO EM DOMINIOS.CPP

ATENSTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS E TIPOS

```
#include <iostream>
#include "dominios.h"

using namespace std;

bool Dominio::setValor(Tipo valor) {
    if (!validar(valor))
        return false;
    this->valor = valor;
    return true;
}

bool DominioA1::validar(Tipo valor){

    cout << "Local para codigo validacao DominioA1" << "\n";

    return true;
}

bool DominioA2::validar(Tipo valor){

    cout << "Local para codigo validacao DominioA2" << "\n";
```

```

        return true;
    }

    bool DominioB1::validar(Tipo valor){

        cout << "Local para codigo validacao DominioB1" << "\n";

        return true;
    }

    bool DominioB2::validar(Tipo valor){

        cout << "Local para codigo validacao DominioB2" << "\n";

        return true;
    }

```

5. Em **fabricas.hpp** e **fabricas.cpp** construir hierarquia de classes fábrica adaptando o seguinte código:

CÓDIGO EM FABRICAS.HPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E ALTERAR NOMES DE DOMÍNIOS E TIPOS

```

#include "dominios.h"

class FabricaDominio {
public:
    virtual Dominio *instanciarDominioA() const = 0;
    virtual Dominio *instanciarDominioB() const = 0;
};

class FabricaDominio1:public FabricaDominio {
public:
    Dominio *instanciarDominioA() const {
        return new DominioA1();
    }

    Dominio *instanciarDominioB() const {
        return new DominioB1();
    }
};

class FabricaDominio2:public FabricaDominio {
public:
    Dominio *instanciarDominioA() const {
        return new DominioA2();
    }

    Dominio *instanciarDominioB() const {
        return new DominioB2();
    }
};

```

6. Na função **main** instanciar domínios usando classe fábrica e interagir com esses domínios:

```

#include "fabricas.h"
#include "dominios.h"

using namespace std;

```

```

int main()
{
    FabricaDominio *fabrica;

    Dominio *dominio;

    // Instanciar fábrica de domínios versão 1 para instanciar domínios dessa versão.

    fabrica = new FabricaDominio1();

    // Instanciar classes da versão 1.

    dominio = fabrica->instanciarDominioA();
    dominio->setValor(100);
    cout << dominio->getValor() << "\n";

    dominio = fabrica->instanciarDominioB();
    dominio->setValor(200);
    cout << dominio->getValor() << "\n";

    // Instanciar fábrica de domínios versão 2 para instanciar domínios dessa versão.

    fabrica = new FabricaDominio2();

    // Instanciar classes da versão 2.

    dominio = fabrica->instanciarDominioA();
    dominio->setValor(300);
    cout << dominio->getValor() << "\n";

    dominio = fabrica->instanciarDominioB();
    dominio->setValor(400);
    cout << dominio->getValor() << "\n";

    return 0;
}

```

7. Selecionar ferramenta online para desenho de diagrama de classes UML.

[Visual Paradigm - Online Productivity Suite](#)

[Draw.io](#)

8. **Criar diagrama UML** que represente as classes e os seus relacionamentos usando ferramenta.

LABORATÓRIO 9

ROTEIRO DE ATIVIDADES

1. Criar projeto **Console application** em C++ usando o Code::Blocks:

File → New → Project

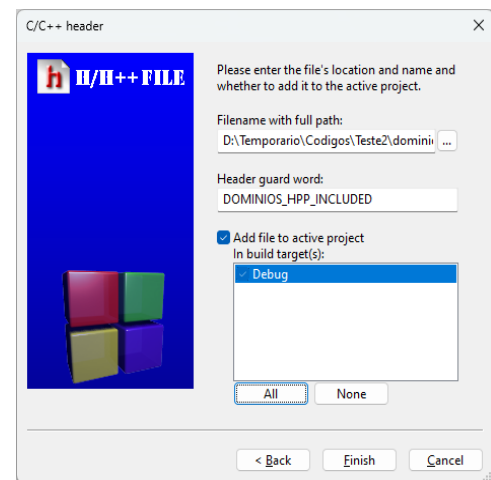
2. Criar arquivo **interfaces.hpp**.

File → New → File... Selecionar C/C++ **header** e informar nome do arquivo **interfaces.hpp**

Selecionar **Add file to active project in build target(s)**

Selecionar **All**

Selecionar **Finish**



3. Implementar **declarações de interfaces** composta por métodos abstratos para o trabalho pratico..
4. **Documentar as interfaces usando ferramenta Doxygen.**

LABORATÓRIO 10

ROTEIRO DE ATIVIDADES

1. Criar projeto **Console application** em C++ usando o Code::Blocks:

File → New → Project

2. Criar arquivos.

Nomes dos arquivos: `domínios.hpp`, `domínios.cpp`, `interfaces.hpp`, `stubs.hpp`, `stubs.cpp`, `controladoras.hpp` e `controladoras.cpp`

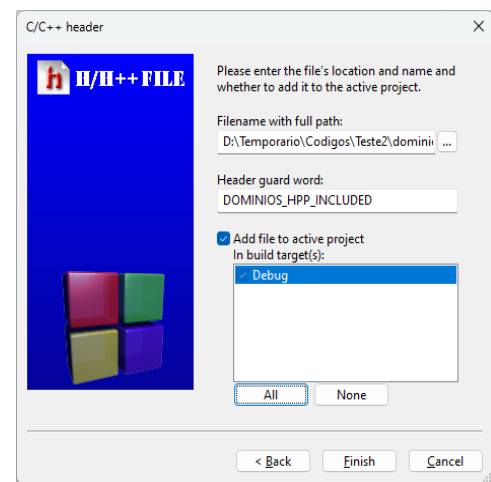
File → New → File... Selecionar **C/C++ header** e informar nome do arquivo com extensão **.hpp**

File → New → File... **Selecionar C/C++ source** e informar nome do arquivo com extensão **.cpp**

Selecionar **Add file to active project in build target(s)**

Selecionar **All**

Selecionar **Finish**



3. Projetar e construir classes **Matricula** e **Senha**.

4. Projetar e construir classe **ILNAutenticacao** contendo método abstrato.

```
bool autenticar(const Matricula&, const Senha&);
```

5. Projetar e construir classe **stub** adaptando o código.

CÓDIGO EM STUBS.HPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E REALIZAR ALTERAÇÕES NECESSÁRIAS.

```
class StubLNAutenticacao:public ILNAutenticacao{  
  
    private:  
        // Definições de valores a serem usados como gatilhos para notificações de erros.  
  
        const static int TRIGGER_FALHA      = 67890;  
        const static int TRIGGER_ERRO_SISTEMA = 78901;  
  
    public:  
        // Declaração de método previsto em interface.  
  
        bool autenticar(const Matricula&, const Senha&);  
  
};
```

CÓDIGO EM STUBS.CPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E REALIZAR ALTERAÇÕES NECESSÁRIAS.

// Definições de constantes.

```
const int StubLNAutenticacao::TRIGGER_FALHA;  
const int StubLNAutenticacao::TRIGGER_ERRO_SISTEMA;
```

// Implementação de método.

```
bool StubLNAutenticacao::autenticar(const Matricula &matricula, const Senha &senha) {  
  
    // Apresentar dados recebidos.  
  
    cout << endl << "StubLNAutenticacao::autenticar" << endl ;  
    cout << "Matricula = " << matricula.getValor() << endl ;  
    cout << "Senha    = " << senha.getValor()    << endl ;  
  
    // Diferentes comportamentos dependendo do valor da matrícula.  
  
    switch(matricula.getValor()){  
        case TRIGGER_FALHA:  
            return false;  
        case TRIGGER_ERRO_SISTEMA:  
            throw runtime_error("Erro de sistema");  
    }  
  
    return true;  
}
```

6. Projetar e construir classe controladora de interface com o usuário adaptando o código.

CÓDIGO EM CONTROLADORAS.HPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E REALIZAR ALTERAÇÕES NECESSÁRIAS.

```
class CntrIUAutenticacao:public IUAutenticacao {  
    private:  
        ILNAutenticacao *cntrLNAutenticacao;    // Referência para servidor.  
    public:  
        bool autenticar(Matricula*);  
        void setCntrLNAutenticacao(ILNAutenticacao*);  
};  
  
void inline CntrIUAutenticacao::setCntrLNAutenticacao(ILNAutenticacao *cntrLNAutenticacao){  
    this->cntrLNAutenticacao = cntrLNAutenticacao;  
}
```

CÓDIGO EM CONTROLADORAS.CPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E REALIZAR ALTERAÇÕES NECESSÁRIAS.

```
bool CntrIUAutenticacao::autenticar(Matricula *matricula) {  
    Senha senha;  
    int entrada;  
  
    // Solicitar matricula e senha.  
  
    while(true) {
```

```

        cout << endl << "Autenticacao de usuario." << endl << endl;

        try {
            cout << "Digite a matricula : ";
            cin >> entrada;
            matricula->setValor(entrada);
            cout << "Digite a senha    : ";
            cin >> entrada;
            senha.setValor(entrada);
            break;
        }
        catch (const invalid_argument &exp) {
            cout << endl << "Dado em formato incorreto." << endl;
        }
    }

    // Solicitar autenticação.

    bool resultado = cntrLNAutenticacao->autenticar(*matricula, senha);

    // Retornar resultado da autenticação.

    return resultado;
}

```

7. Na função **main** instanciar controladora e stub usando os métodos adequados adaptando o código:

CÓDIGO EM MAIN.CPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E E REALIZAR ALTERAÇÕES NECESSÁRIAS.

```

int main()
{
    // Instanciar classe controladora e classe stub.

    IUAutenticacao *cntrIUAutenticacao = new CntrIUAutenticacao();
    ILNAutenticacao *stubLNAutenticacao = new StubLNAutenticacao();

    // Ligar instância de controladora a instância de stub.

    cntrIUAutenticacao->setCntrLNAutenticacao(stubLNAutenticacao);

    bool resultado;

    Matricula matricula;

    while(true){

        // Simular apresentação de tela inicial do sistema.

        cout << endl << "Tela inicial de sistema." << endl;

        try{
            // Soliciatar serviço de autenticação.

            resultado = cntrIUAutenticacao->autenticar(&matricula);
        }
        catch(const runtime_error &exp){
            cout << "Erro de sistema." << endl;
            break;
        }
    }
}

```

```

        // Criticar resultado da autenticação.

        if(resultado) {
            cout << endl << "Sucesso autenticacao." << endl;
            cout << endl << "Matricula = " << matricula.getValor() << endl;
            break;
        }
        else {
            cout << endl << "Erro autenticacao." << endl;
            break;
        }
    }

    // Destruir instância de controladora e instância de stub.

    delete cntrIUAutenticacao;
    delete stubLNAutenticacao;

    return 0;
}

```

LABORATÓRIO 11

ROTEIRO DE ATIVIDADES

1. Reusar projeto anterior.
2. Criar arquivos **containers.hpp** e **containers.cpp**.

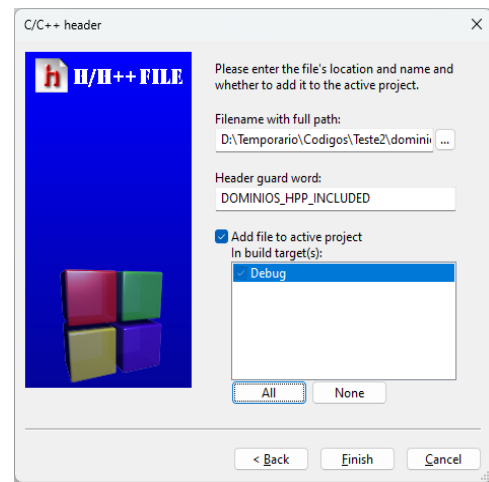
File → New → File... Selecionar C/C++ **header** e informar nome do arquivo **containers.hpp**

File → New → File... Selecionar C/C++ **source** e informar nome do arquivo **containers.cpp**

Selecionar **Add file to active project in build target(s)**

Selecionar **All**

Selecionar **Finish**



3. **Projetar e construir classe container** usando template **vector**, **list** ou **map** e adaptando o código:

CÓDIGO EM CONTAINERS.HPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E REALIZAR ALTERAÇÕES NECESSÁRIAS.

```
class ContainerUsuario{
private:
    list<Usuario> container;
public:
    bool incluir(Usuario);
    bool remover(Matricula);
    bool pesquisar(Usuario*);
    bool atualizar(Usuario);
};
```

CÓDIGO EM CONTAINERS.CPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E REALIZAR ALTERAÇÕES NECESSÁRIAS.

```
bool ContainerUsuario::incluir(Usuario usuario){
    for(list<Usuario>::iterator elemento = container.begin(); elemento != container.end(); elemento++){
        if (elemento->getMatricula().getValor() == usuario.getMatricula().getValor()){
            return false;
        }
    }
    container.push_back(usuario);
    return true;
}
```

```

bool ContainerUsuario::remover(Matricula matricula){
    for(list<Usuario>::iterator elemento = container.begin(); elemento != container.end(); elemento++){
        if (elemento->getMatricula().getValor() == matricula.getValor()){
            container.erase(elemento);
            return true;
        }
    }
    return false;
}

bool ContainerUsuario::pesquisar(Usuario* usuario){
    for(list<Usuario>::iterator elemento = container.begin(); elemento != container.end(); elemento++){
        if (elemento->getMatricula().getValor() == usuario->getMatricula().getValor()){
            *usuario = *elemento;
            return true;
        }
    }
    return false;
}

bool ContainerUsuario::atualizar(Usuario usuario){
    for(list<Usuario>::iterator elemento = container.begin(); elemento != container.end(); elemento++){
        if (elemento->getMatricula().getValor() == usuario.getMatricula().getValor()){
            *elemento = usuario;
            return true;
        }
    }
    return false;
}

```

4. Na função **main** instanciar container e interagir usando os métodos adequados.

CÓDIGO EM MAIN.CPP

ATENTAR PARA A POSIÇÃO DE CADA LINHA E REALIZAR ALTERAÇÕES NECESSÁRIAS.

```

ContainerUsuario container;

resultado = container.incluir(usuario_1);

if(!resultado){
    cout << "Erro na inclusao" << endl;
}
else {
    cout << "Sucesso na inclusao" << endl;
}

```

LABORATÓRIO 12
ROTEIRO DE ATIVIDADES

1. **Demonstrar compilação e execução de código** integrante do trabalho entregue para correção.